

P40

Dropout: una forma simple de evitar el sobreajuste en redes neuronales

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov · 2014 · JMLR
15(56):1929–1958

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P40 — Dropout

Arquitectura y entrenamiento · Apagar unidades al azar equivale a entrenar un ensamblado exponencial de subredes que comparten pesos.

Nivel: L2 · Motor: dropout · Notebook: P40_dropout.ipynb · Anexo: probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	<i>Dropout: A Simple Way to Prevent Neural Networks from Overfitting</i>
Autoría	Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov
Año	2014
Venue	<i>Journal of Machine Learning Research</i> 15(56), 1929–1958
Fuente primaria	jmlr.org/papers/v15/srivastava14a
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las redes grandes memorizaban el conjunto de entrenamiento. La receta conocida contra eso — promediar muchos modelos entrenados por separado — funciona muy bien y es **inviable** con redes profundas: entrenar diez redes cuesta diez veces más, y servir las también.

Había además un problema más sutil que el sobreajuste clásico: la **co-adaptación**. Una unidad podía desarrollar una función que solo era útil en presencia de otra unidad concreta. El conjunto funcionaba en entrenamiento y era frágil ante cualquier cambio.

3. Propuesta

En cada paso de entrenamiento, poner a cero cada unidad con probabilidad p , de forma independiente. Ninguna función puede depender de que una unidad concreta esté presente, porque a menudo no lo estará.

La interpretación que da el paper: con n unidades hay 2^n subredes posibles, y entrenar con dropout es entrenar ese **ensamblado exponencial** con pesos compartidos. En inferencia se usan todas las unidades con los pesos escalados, lo que aproxima el promedio de todas esas subredes.

4. Intuición sin fórmulas

Si en un equipo cada tarea depende de dos personas concretas, el día que una falte no se hace nada. Si todos pueden cubrir varias tareas, el equipo aguanta. Dropout obliga a lo segundo haciendo faltar a gente al azar

cada día.

Dónde deja de funcionar la analogía: un equipo real reorganiza el trabajo conscientemente. La red no «decide» ser redundante: es que las representaciones redundantes son las únicas que reciben gradiente útil de forma consistente.

5. Matemática mínima

Entrenamiento: $m \sim \text{Bernoulli}(1 - p)$ una máscara nueva por paso
 $\tilde{h} = m \odot h$

Inferencia : $h_{\text{test}} = (1 - p) \cdot h$ se usan todas, escaladas

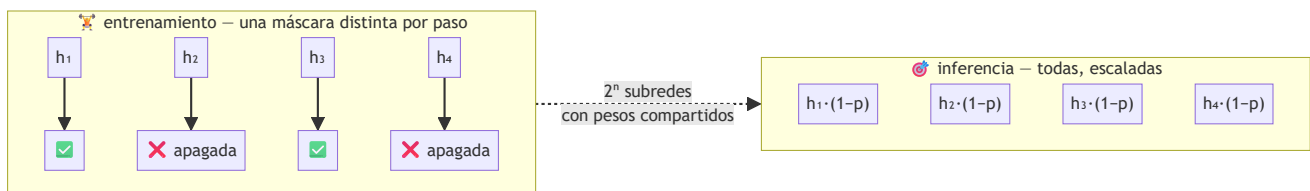
Equivalente y más usado (inverted dropout):
 $\tilde{h} = (m \odot h) / (1 - p)$ durante el entrenamiento
 $h_{\text{test}} = h$ sin cambio en inferencia

El escalado no es cosmético: sin él, la magnitud esperada de las activaciones en inferencia es $1/(1-p)$ veces la del entrenamiento, y el modelo trabaja en un régimen distinto del que aprendió.

Por qué premia la redundancia, con $p = 0,5$ y unidades independientes:

una función que necesita 2 unidades concretas → disponible el 25 % de los pasos
una repartida entre 3 unidades cualesquiera → disponible el 87,5 %

6. Arquitectura o flujo



7. Qué observar en el paper original

- La **interpretación como ensamblado** y por qué el escalado en inferencia aproxima el promedio geométrico de las subredes. Es una aproximación, y el paper lo dice.
- Los experimentos sobre **qué aprenden las unidades** con y sin dropout: las representaciones visualizadas son más localizadas e interpretables con dropout.
- La discusión sobre el **valor de p** : 0,5 en capas ocultas y valores menores en la entrada, y por qué.
- Que se evalúa en **muchos dominios** —visión, voz, texto, genómica—: la afirmación es de generalidad, no de un buen resultado puntual.

8. Evidencia y resultados

Mejoras consistentes en conjuntos de visión, reconocimiento de voz, clasificación de documentos y datos biológicos, comparando la misma arquitectura con y sin dropout.

Las cifras por conjunto están en el artículo. Verificarlas allí; lo transferible es el mecanismo y el patrón de mejora, no los números de 2014.

La miniatura de este eje cuantifica el argumento de la co-adaptación: una función que depende de dos unidades concretas está disponible una cuarta parte de las veces, y una repartida entre tres, casi siempre.

9. Impacto

- Fue la regularización por defecto en deep learning durante media década.
- Su interpretación como ensamblado influyó en toda una línea de trabajo sobre regularización estocástica.
- Y hoy se usa **menos** en visión y en Transformers grandes, donde la escala de datos y otras técnicas cumplen ese papel. Es un buen recordatorio de que las recetas del campo caducan.

10. Limitaciones

1. **Ralentiza la convergencia:** hacen falta más épocas para el mismo resultado.
2. **Interacción mal entendida con la normalización por lotes**, que produjo fallos sutiles durante años al combinarlas.
3. **La equivalencia con el ensamblado es aproximada**, no exacta, y el paper lo declara.
4. **Menos útil con muchos datos:** si el conjunto es grande, hay poco sobreajuste que evitar.
5. **Un hiperparámetro más** (p) que ajustar por capa.
6. **Poco efectivo en capas convolucionales** frente a las densas, por la compartición de pesos.

11. Errores comunes

Error	Corrección
«Dropout añade ruido para regularizar»	Cambia qué representaciones son rentables : penaliza la dependencia de unidades concretas.
Dejarlo activo en inferencia	La misma entrada daría salidas distintas y con magnitud equivocada. Es un fallo clásico en producción.
Aplicar el escalado dos veces	Existen dos convenciones (escalar en inferencia o dividir en entrenamiento). Usar ambas rompe el modelo.
«Siempre conviene ponerlo»	Con muchos datos o arquitecturas modernas puede no aportar y sí ralentizar.
«Es equivalente a entrenar 2^n redes»	Es una aproximación al promedio de esas subredes, no una equivalencia exacta.

12. Relación con trabajos anteriores

- **Po4 AlexNet (2012)** — lo usa en sus capas densas; este artículo es el desarrollo completo de la idea.
- **Hinton et al. (2012)** — la versión preliminar. [arXiv:1207.0580](#)
- **Bagging y ensamblados** — el marco clásico que se aproxima sin pagar su coste.

13. Relación con trabajos posteriores

- **P43 BatchNorm (2015)** — otra forma de estabilizar, con la que interactúa de forma no trivial.
- **DropConnect, DropPath, stochastic depth** — variantes sobre la misma idea.
- **Regularización moderna** — aumento de datos, decaimiento de pesos y escala, que en muchos casos lo han sustituido.

14. Notebook asociado

P40_dropout.ipynb

Qué implementa: el cálculo de disponibilidad de una función co-adaptada frente a una redundante, el conteo de subredes y la demostración del escalado en inferencia.

Qué NO implementa: no hay red, ni datos, ni entrenamiento. Se cuentan probabilidades de máscara para exhibir el argumento.

```
ai-evolution paper-lab P40 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe qué ocurre en entrenamiento y qué en inferencia.
Explicar	Explica por qué dropout premia las representaciones redundantes.
Aplicar	Calcula la disponibilidad de una función que necesita 4 unidades concretas con $p=0,5$.
Analizar	¿Por qué es menos efectivo en capas convolucionales?
Evaluar	Un modelo «funciona peor en producción» sin causa aparente. ¿Qué comprobarías primero?
Crear	Diseña un experimento que distinga el efecto de ensamblado del de anti-co-adaptación.

16. Autoevaluación

1. ¿Qué es la co-adaptación y por qué es un problema?
2. ¿Cuántas subredes hay con n unidades?
3. ¿Por qué hace falta escalar y qué pasa si no se hace?
4. ¿Cuáles son las dos convenciones de escalado?
5. ¿Por qué ralentiza la convergencia?
6. ¿Por qué se usa menos hoy en modelos grandes?
7. ¿En qué sentido la equivalencia con un ensamblado es aproximada?

17. Respuestas esperadas

1. Que una unidad desarrolle una función útil solo en presencia de otra concreta. Produce soluciones frágiles que dependen de una configuración exacta del resto de la red.
2. 2^n , una por cada subconjunto de unidades activas. Todas comparten los mismos pesos.
3. Porque en entrenamiento solo está activa la fracción $1-p$ de las unidades, así que la suma esperada es menor. Sin escalar, en inferencia las activaciones son $1/(1-p)$ veces mayores que las vistas al entrenar.
4. Escalar por $(1-p)$ en inferencia, o dividir por $(1-p)$ durante el entrenamiento (inverted dropout). Se elige una; aplicar las dos rompe el modelo.
5. Porque cada paso entrena una subred distinta y con gradiente más ruidoso: hacen falta más pasos para el mismo progreso efectivo.
6. Porque con conjuntos enormes hay poco sobreajuste, y otras técnicas —aumento de datos, decaimiento de pesos, la propia escala— cubren ese papel con menos coste.
7. El escalado en inferencia aproxima el promedio geométrico de las predicciones de las subredes, pero no lo calcula exactamente. El propio paper lo presenta como aproximación.

18. Fuentes primarias

- Srivastava, N. et al. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. **JMLR** 15(56), 1929–1958. jmlr.org/papers/v15/srivastava14a · consultado 2026-08-16.
- Hinton, G. E. et al. (2012). *Improving neural networks by preventing co-adaptation of feature detectors*. arXiv:1207.0580 · consultado 2026-08-16.

Evaluación — P40 · Dropout: una forma simple de evitar el sobreajuste en redes neuronales

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Dropout: A Simple Way to Prevent Neural Networks from Overfitting* (2014, JMLR 15(56):1929–1958) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P40_dropout.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).